Laboratory Work No. 3 – The Command Line: Shell, Pipes and Text Filters

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 3: The Command Line: Shell, Pipes and Text Filters

Track A: The Real OS, from the Outside (Linux)

Elaborated by: st. gr. FAF-241, Sitarciuc Dmitri

Verified by: Reițman Patricia

Chișinău – 2026

---

1. Introduction

The purpose of this laboratory work was to understand what the Bash shell does with a command line before a program starts: finding commands, expanding patterns and variables, handling quotes, redirecting input and output, chaining commands and running background jobs. I also answered questions about a log file with one-line pipelines and wrote my first Bash script.

The work was done in GitHub Codespaces (Ubuntu 24.04 in a container), in the folder `~/os-lab3`. To keep a clean record, I put the commands into a script, `lab3.sh`, where a small helper function prints each command with a `$` in front and then runs it. The outputs below are from that run. Interactive features (Tab completion and history search) cannot be recorded by a script, so they are described by how Bash handles them.

2. Data Preparation

The log file for the exercises was generated with the loop from the lab sheet:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i %
5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log
wc -l access.log
```

```
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

Each line has five fields: date, time, user, status and page. The file has exactly 500 lines. Because it is built from a formula (every 5th request fails, users cycle through 7 values, pages through 4), the answers in Part 6 can also be checked by arithmetic.

3. Move Fast, Find Help

Tab completion and history search are handled by Readline, the line editor built into Bash. When a typed prefix matches exactly one name, Tab completes it, so `ls /usr/sh` becomes `ls /usr/share/` and `cd /et` becomes `cd /etc/`. When several names match, a second Tab lists them. Ctrl+R opens `(reverse-i-search)`, where typing part of an old command, such as `make`, shows the most recent matching command.

Then I checked how Bash finds commands:

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
$ type echo
echo is a shell builtin
$ which ls
/usr/bin/ls
$ nosuchprog; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
$ ls --help | grep -- " -t "
  -t                         sort by time, newest first; see --time
```

`echo $PATH | tr ":" "\n"` printed 36 directories. The first one belongs to VS Code (`/vscode/bin/.../remote-cli`), followed by many language tools installed in Codespaces (Python, Node.js, Java, Go, Ruby, PHP, .NET), and only then the standard

`/usr/local/bin`, `/usr/bin` and `/bin`. Two directories appear twice: `/home/codespace/.local/bin` and `/usr/local/rubies/current/bin`.

3.1. Observations

1. `cd` and `echo` are shell built-ins; `ls` is an external program at `/usr/bin/ls`. `cd` must be a built-in: the current directory belongs to a process, and an external program runs as a child process, so it could only change its own directory, not the shell's. In my script `type ls` showed the file path. In an interactive terminal Ubuntu also defines an alias `ls --color=auto`, but aliases are not loaded in non-interactive scripts.
2. `PATH` has 36 entries (34 different ones); `ls` was found in `/usr/bin`.
3. A command that does not exist returns exit code 127.
4. The `ls` option that sorts by modification time is `-t`.
5. Ctrl+R searches backwards through the command history: you type part of an old command and Bash shows the latest match, which you can run with Enter.

3.2. Interesting Command Output

The long `PATH` was the most surprising. Bash searches these directories in order, so a program in an early directory hides one with the same name later in the list. In Codespaces dozens of language tools are placed before the system directories, so, for example, a `python` in `/home/codespace/.python/current/bin` would be found before one in `/usr/bin`.

4. Shell Expansion and Quoting

In a test folder I created six files, one of them with a space in its name:

```
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
```

Before running each command I wrote down what I expected, then compared it with the real output. **[TODO: fill in the "My prediction" column with your own guesses.]**

4.1. Prediction and Results

Command	My prediction	Real output	Explanation
`echo *.c`		`a.c b.c`	Bash replaces the glob with matching file names before `echo` starts
`echo file?.txt`		`file1.txt file2.txt`	`?` matches exactly one character
`echo [a-b]*`		`a.c alfa beta.txt b.c`	`[a-b]` matches a first letter a or b
`echo *.xyz`		`*.xyz`	No match, so Bash leaves the pattern as it is
`ls alfa beta.txt`		two errors: cannot access `alfa`, cannot access `beta.txt`	The unquoted space splits the name into two arguments
`ls "alfa beta.txt"`		`alfa beta.txt`	Quotes keep it one argument
`echo '$HOME' "$HOME" \$HOME`		`$HOME /home/codespace $HOME`	Single quotes: literal; double quotes: expanded; backslash: escapes `$`
`echo {1..5}`		`1 2 3 4 5`	Brace expansion
`echo $((7*6))`		`42`	Arithmetic expansion
`echo "today is $(date +%A), I am $(whoami)"`		`today is Friday, I am codespace`	Command substitution inside double quotes
`touch log{1..3}.txt && ls log*`		`log1.txt log2.txt log3.txt`	Braces create three names; `&&` runs `ls` after success

4.2. Observations

1. `echo` does not look at files at all. Bash expands `*.c` into `a.c b.c` first, and `echo` just receives two ready arguments.

2. `ls alfa beta.txt` fails because Bash splits words on spaces, so `ls` gets two names that do not exist. Two correct ways: `ls "alfa beta.txt"` or `ls alfa\ beta.txt`.
3. Single quotes keep every character literally. Double quotes keep spaces together but still allow `$VAR`, `$(command)` and `$((math))`.
4. In some terminals `ls` shows the file as `'alfa beta.txt'` with quotes. The quotes are only for display, so the space is visible; they are not part of the name.

5. Redirection, Chaining and Jobs

```
$ ls /etc /nope > out.txt 2> err.txt
$ tail -2 out.txt
xml
zsh
$ cat err.txt
ls: cannot access '/nope': No such file or directory

$ ls /etc /nope > all.txt 2>&1
$ grep nope all.txt; tail -2 all.txt
ls: cannot access '/nope': No such file or directory
xml
zsh

$ ls /etc /nope 2>&1 > all2.txt
ls: cannot access '/nope': No such file or directory

$ wc -l < access.log
500
$ wc -l access.log
500 access.log

$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
$ true && echo "after success"
after success
$ false ; echo "always runs"
always runs

$ sleep 100 &
$ jobs
[1]+  Running                 sleep 100 &
$ kill %1
$ sleep 0.2; jobs
$
```

5.1. Observations

1. With `> out.txt 2> err.txt`, the normal listing of `/etc` went to `out.txt` and the error about `/nope` went to `err.txt`. Nothing appeared on screen.
2. Order matters. In `> all.txt 2>&1`, stdout goes to the file first, then stderr is pointed to the same place, so both are in the file (the error is in `all.txt`). In `2>&1 > all.txt`, stderr is first pointed to where stdout is at that moment, the terminal, and only then stdout moves to the file, so the error still appears on screen.
3. `wc -l < access.log` gets the data on standard input and does not know the file's name, so it prints only `500`. `wc -l access.log` opens the file itself and prints the name too.
4. `;` always runs the next command, `&&` only if the previous one succeeded (exit code 0), `||` only if it failed. `ls /nope` failed with code 2, so `||` printed the message.
5. `&` started `sleep 100` in the background as job 1, `jobs` showed it as Running, `kill %1` stopped it, and the second `jobs` (run after a short pause so the kill could finish) printed nothing, because no jobs were left.

6. Answering Questions with One-liners

All answers were saved in `oneliners.sh` and run with `bash -v oneliners.sh`, which prints each line before running it.

Q1. How many requests failed? 100.

```
grep -c FAIL access.log
```

Q2. How many different pages, and which? 4: `/page0`, `/page1`, `/page2`, `/page3`.

```
cut -d' ' -f5 access.log | sort -u | awk '{print} END{print NR, "different pages"}'
```

Q3. Requests per page? 125 each.

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Q4. Which user has the most failures? A tie: `user3` and `user5`, 15 each.

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1} $1==m'
```

The `awk` part remembers the count of the first (largest) line and prints every line with the same count, so a tie is not hidden.

Q5. Requests by user3, and how many failed? 72 requests, 15 failed.

```
grep -w user3 access.log | awk '{t++; if($4=="FAIL") f++} END{print t"
requests, "f" failed"}'
```

Q6. Last 3 failed requests (time and user): `10:10 user0`, `10:15 user5`, `10:20 user3`.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

Q7. Login shells in /etc/passwd: 20 accounts use `/usr/sbin/nologin`, 2 use `/bin/bash`, 1 uses `/bin/sync`.

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Most accounts are system accounts for services that should never log in, so they get `nologin`.

Q8. Why do `ls /etc | wc -l` and `ls -l /etc | wc -l` differ by one? They printed 163 and 164. The long format adds one header line, `total 824`, the disk space used by the listed files in 1 KB blocks.

```
ls /etc | wc -l ; ls -l /etc | wc -l ; ls -l /etc | head -1
```

Bonus. Failed requests where the minute ends in 0: 50.

```
grep -E ' 10:[0-5]0 user[0-9] FAIL ' access.log | wc -l
```

6.1. Checking the Answers

The results agree with the formula that created the log. Every 5th line fails, so 500 / 5 = 100 failures. Pages cycle through 4 values, so 500 / 4 = 125 each. A minute ends in 0 only on lines where `i` is a multiple of 10, and those are always failures (multiples of 5), so 500 / 10 = 50. The per-user failure counts in Part 7 also add up: 5 × 14 + 2 × 15 = 100.

7. My First Bash Script

The last task turned the one-liners into a reusable script, `report.sh`, that takes a log file name, checks it exists, and prints totals and failures per user.

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
  echo "no such file: $log" >&2
  exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for i in $(seq 0 6); do
  n=$(grep -c "user$i FAIL" "$log")
  echo "user$i: $n failures"
done
```

7.1. Test

```
$ ./report.sh access.log
./report.sh: Permission denied

$ chmod +x report.sh && ls -l report.sh
-rwxr-xr-x 1 codespace codespace 305 Oct 23 08:35 report.sh

$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures

$ ./report.sh nope.log; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

7.2. Observations

1. `#!/bin/bash` (the shebang) tells the kernel which interpreter should run the file. The first run failed with `Permission denied` because a new file has no execute permission; after `chmod +x` the permissions became `-rwxr-xr-x` and the script ran.
2. The script is run as `./report.sh`, not `report.sh`, because the current directory is not in `PATH`. `./` gives the exact path to the file.

3. `>&2` sends the error message to standard error instead of standard output, and `exit 1` returns a non-zero exit code, so another script (or `&&` / `||`) can detect the failure. The test confirmed exit code 1 for a missing file.

8. Conclusion

This laboratory work showed that the shell does a lot of work before any program starts. Bash finds the program through `PATH` (or runs a built-in like `cd`), expands globs, variables, braces, arithmetic and command substitutions, removes quotes, and sets up redirections. The program only receives the final list of arguments.

I also saw that small filters (`grep`, `cut`, `sort`, `uniq`, `awk`, `wc`) connected with pipes can answer real questions about data in one line, and that the answers can be checked against how the data was built. Finally, I wrote `report.sh`, a script that validates its input, reports errors on stderr and returns a proper exit code.